PRACTICAL - 8

Implementation of Graph Searching using DFS and BFS

Aim of the Practical:

To implement Depth First Search (DFS) and Breadth First Search (BFS) for graph traversal and analyze their time and space complexities.

Algorithm:

1. Represent the graph using an adjacency list.
2. For BFS, use a queue and visit vertices level by level.
3. For DFS, use recursion and visit an unvisited adjacent vertex before backtracking.
4. Maintain a visited array so that every vertex is processed only once.

Program:

```
#include <iostream>
#include <vector>
#include <queue>
using namespace std;
void bfs(int start, const vector<vector<int>>& adj) {
    vector<bool> visited(adj.size(), false);
    queue<int> q;
    visited[start] = true;
    q.push(start);
    while (!q.empty()) {
        int u = q.front(); q.pop();
        cout << u << " ";
        for (int v : adj[u]) {
            if (!visited[v]) {
                visited[v] = true;
                q.push(v);
            }
        }
    }
}
void dfs(int u, const vector<vector<int>>& adj, vector<bool>& visited) {
    visited[u] = true;
    cout << u << " ";
    for (int v : adj[u])
        if (!visited[v]) dfs(v, adj, visited);
}
int main() {
    vector<vector<int>> adj = {
        {1,2}, {0,3,4}, {0,4}, {1}, {1,2}
    };
    cout << "BFS: ";
    bfs(0, adj);
    vector<bool> visited(adj.size(), false);
    cout << "\nDFS: ";
    dfs(0, adj, visited);
}
```

Output:

```
BFS: 0 1 2 3 4
DFS: 0 1 3 4 2
```

Time Analysis:

Step-by-Step Derivation:

Let V be the number of vertices and E be the number of edges.

With an adjacency list, initializing the visited array requires $\Theta(V)$.

Each vertex is inserted/removed from the BFS queue at most once, or entered by DFS at most once.

$$\text{Vertex-processing cost} = \Theta(V)$$

Name: SHAIK ASHWAK BASHA **Enrollment No: 92460118522**

Across all adjacency lists, every undirected edge appears twice (once at each endpoint), so all neighbor scans together require Θ(E).

$$\text{Edge-scanning cost} = \Theta(E)$$

$$T(V,E) = \Theta(V) + \Theta(E) = \Theta(V + E)$$

BFS stores the queue and visited array; DFS stores the visited array and recursion stack. Both can require Θ(V) auxiliary space.

Final Time Analysis:

- BFS Time: Θ(V + E).
- DFS Time: Θ(V + E).
- BFS Space: Θ(V).
- DFS Space: Θ(V).

Result:

BFS and DFS were implemented successfully, and their linear graph-traversal complexity was derived from vertex and edge processing.